

Clean Code - Emergent Design Notes

"Good design emerges from simple rules."

The 4 Rules of Simple Design (Kent Beck)

1. Runs all the tests 2. No duplication 3. Expresses intent 4. Minimal classes and methods

1. Runs All the Tests

A system must be testable. Testing improves design quality.

Bad Example (Hard to Test)

```
class PaymentService:
    def process(self):
        db = MySQLDatabase()
        api = StripeAPI()
        return api.charge(db.get_user())
```

Good Example (Testable with DI)

```
class PaymentService:
    def __init__(self, db, api):
        self.db = db
        self.api = api

    def process(self):
        return self.api.charge(self.db.get_user())
```

2. No Duplication

Duplication increases complexity and maintenance cost.

Bad Example

```
def is_empty(list):
    return len(list) == 0

def has_items(list):
    return len(list) > 0
```

Good Example

```
def is_empty(list):
    return len(list) == 0

def has_items(list):
    return not is_empty(list)
```

3. Expressive Code

Code should clearly express intent.

Bad Example

```
def calc(x, y):  
    return x * y * 0.1
```

Good Example

```
def calculate_discount(price, quantity):  
    return price * quantity * 0.1
```

4. Minimal Classes & Methods

Avoid over-engineering and unnecessary abstraction.

Bad Example

```
class IUserService: pass  
class UserServiceImpl(IUserService): pass  
class UserServiceFactory: pass
```

Good Example

```
class UserService:  
    def get_user(self): pass
```

Refactoring Example

Extract duplication into reusable function.

```
def replace_image(new_image):  
    image.dispose()  
    gc.collect()  
    return new_image
```

Template Method Pattern

```
class VacationPolicy:  
    def accrue(self):  
        self.base()  
        self.legal()  
        self.apply()  
  
    def base(self): pass
```

```
def apply(self): pass
def legal(self): raise NotImplementedError
```

Key Takeaways

- Tests drive good design - Remove duplication aggressively - Write expressive code - Avoid over-engineering - Refactor continuously